Configuration transport and state

Use this page for whole-document replacement or interactive-state recovery.
Domain pages own their fields. Tool descriptions define invocation parameters; they do not replace required reading or procedures. Reuse complete contracts already in context.

Configuration surface

Project config is platform-stored; Studio overrides only runtime.port in Session-local state. The config plane is distinct
from the application runtime API (`MANUS_API_URL` / `MANUS_API_KEY`), described in service API. There is no local `webdev.json` , `webdev.config_save` , or `webdev.config_view` mechanism.

A skill endpoint such as `PUT <config-plane>/config/runtime` denotes the registered config tool with that method, project-relative path `config/runtime` , and JSON body. The task's binding fixes the project; paths cannot target another project. Authentication and the skill version are supplied by the tool. Query parameters belong in `query` , never in the path.

Surface	Meaning
`GET config`	Stored declaration and current derived state.
`PUT config/{domain}`	Replace one of `features` , `build` , `deploy` , `resources` , `routes` , `pwa` , `runtime` , `git` , `hosting` , `preview` .
`PUT config`	Atomic project replacement; Studio preserves both port values.
`PUT` / `DELETE secrets/{NAME}`	Advanced declaration/revision or withdrawal; new protected input uses the registered request-secrets tool.
`POST integrations/shopify/enable`	Store selection; Shopify.
`POST integrations/stripe/enable`	Platform Stripe enablement; payments.
`GET env`	Discover injected runtime key names; values are redacted.
`runtime/post-edit`	LSP registration: diagnostics.
`GET logs`	Published-site logs; diagnostics.
`GET infra/overview`	Current infrastructure observations; no config revision or runtime load receipt.

`POST publish`	Publish the latest checkpoint as a background job; its final result returns automatically. Do not poll. See Git/checkpoints.

`GET config` returns `ok`, `revision`, `config`, secret availability, and applicable derived fields such as `publishing`, `git_repository`, `stripe`, `runtime_endpoints`, `preview_origin`, and `preview_device`. Derived fields are not declaration fields. `GET env` returns key names with `<redacted>` placeholders; user-declared secret keys are excluded. No sandbox config response
returns a stored protected value.

`publishing.auto_publish` is a read-only user preference outside `config`; config writes cannot enable it. It controls automatic publication after a new checkpoint. An explicit `POST publish` is independent of this preference in both Cloud and Studio; a user request to publish does not require enabling auto-publish. Follow Git checkpoints.
No config-plane endpoint creates a project. Its path catalog is the registered tool; external HTTP `_list` discovery is not an internal MCP endpoint.

Write preparation

Before a domain write, `GET config`, apply only the intended change to the current state, and resubmit the complete domain with fields not being changed preserved. Write one domain per
call. Submit canonical-Git changes and secret-input candidates separately from unrelated changes; do not use a full-document import to bypass their user-input lifecycle.

Before writing a domain, read its precise contract; do not guess a body from a legacy README.
Read secrets before adding or revising a declaration, and use the registered request-secrets tool when the user must supply its value. Read payments before writing payment integration code, not only when calling enable.

Never send a per-request computed response
to the `static` target: a static miss cannot stand in for the dynamic application. Read hosting before modifying resources or residency.

When stale or corrupted image output plausibly comes from the build cache, GET the complete
config, preserve it, set the **top-level** `buildCache: false`, and PUT the complete document. There is no buildCache domain endpoint. This asks for a fresh image build on the next publish;

it is not a development-cache reset or a reason to repeat publishes without inspecting failures.

Complete replacement

A domain write's body is the domain fragment, for example `{"runtime": {"port": 3000}}`.
It supplies that domain's complete new state; omitted fields in that domain are not preserved.
An empty body clears an optional domain, subject to the resulting document's validity and one-way capability rules. The required runtime domain cannot be cleared.

In Studio, keep `runtime.port` from `GET config` unchanged in whole-document writes. The tool
preserves the project's port and Session override, while replacing other project fields.
Change the local port separately with `PUT config/runtime`. The whole-document structure is:

Key	Owner
`version`	Required literal `1`, the declaration format version, not a revision counter.
`runtime`	Required; runtime.
`features`	Optional; features.
`build`	Optional; static publishing.
`deploy`, `buildCache`	Optional; container publishing.
`resources`, `hosting`	Optional; hosting.
`preview`	Optional; runtime.
`routes`	Optional; published routes.
`git`	Optional; canonical Git.
`integrations`	Optional; payments.
`secrets`	Key-to-declaration map, default `{}`; secrets.

Unknown fields are rejected. Whole replacement validates the resulting document atomically;
for example, a declaration can carry `features.server` and its dependent `deploy` together.
It cannot combine a canonical-Git change with unrelated changes, a new secret-input candidate

with non-secret changes. Managed integrations such as Stripe and Shopify must be enabled through their dedicated routes; a raw config replacement is not an enable operation. Preserve existing enabled integrations when replacing the document. These operations have user-controlled settlement or provisioning state of their own.

If an old declaration causes a compatibility failure, consult
legacy configuration background.

Response envelope

A successful write is `{"ok": true, "revision": N}`. The revision acknowledges durable storage, not published behavior. An unchanged accepted declaration can return the current revision without applying another change.

A rejection is `{"ok": false, "error": {"code": "...", "field_errors": [...], "note": "..."}}`.
It is an ordinary tool result, not necessarily a transport failure. Config writes are stored or rejected as a whole, but a rejected write can still create pending card state below.
`error.code`, `error.reason_code`, and `error.field_errors[].code` are distinct positions; field errors include their affected `path`. Body validation reports a bounded subset of errors; logs-query validation reports all invalid query parameters. Errors is a code-to-contract index, not a separate retry workflow.

`404 resource_not_found` intentionally does not distinguish an absent resource from one the caller cannot access. `revision_conflict` describes changed config state except when a more specific `reason_code` identifies a pending operation.

Handling a rejected operation

Web domain PUTs commit only the requested domain, and secret declarations only the named key.
Unrelated domain updates do not cause revision conflicts. Whole-document PUT retains its replacement semantics. Read `error.reason_code` first: a `revision_conflict` can indicate a whole-document conflict, a changed dependency of a pending operation, or its lifecycle state.
Inspect the affected state before resubmitting; do not blindly replay external effects. `secret_input_in_progress` is a pending card, not revision movement; do not loop on config reads and writes to clear it.

For `validation_failed`, correct the affected declaration before resubmitting. Body errors can be capped; query errors can be complete. The server does not retry rejected writes for you.

A successful durable write needs no extra generic settlement call or poll. On runtime load failure, inspect and correct the reported cause, make one GET config retry, and require `runtime_sync` to be `applied` or `noop` before proceeding with the new values. If unresolved,

report the failure and active/desired distinction. Do not use a responding old process to claim that the new revision loaded.

On `binding_replaced`, do not re-init or re-attach. Retry the failing operation once after the next runtime reconcile, then report persistent failure and its request id. Other typed errors have their concrete retry/stop/owner actions in errors; use that action rather than treating every failure as permission for another initialization.

For a reported Skill-version mismatch, see the version background.

Interactive cards

`action_url` identifies a server-owned card or link. Its presence alone does not mean the Session must stop: Host effects and Widget notifications control input waiting and resumption.
Never end the task solely because `action_url` is present. The Stripe status/claim card is non-blocking: continue implementation without waiting for a later claim. For an actual `confirmation_required` owner-input request, stop the turn and wait for that confirmation; do not keep submitting it. After completion, GET config to verify the applied state.
`secret_values_required` uses the registered secret tool's stop/wait instruction. If another input card is pending, do not ask for values in chat or retry a blocked write. With `blocks_config_writes:false`, unrelated scoped writes may proceed; conflicting keys, dependent changes and whole-document replacement wait for completion or withdrawal. `true` blocks all writes.

Reserved hosting is the `always_on` confirmation case. Its `409 confirmation_required` has `reason_code: hosting_confirmation_required`, an operation card, and an agent pause. The Widget,
not a second verbal question, obtains the product confirmation. On its confirmed notification,
the original `PUT config/hosting` is already applied: GET `config`, then `infra/overview`, and do not repeat the PUT or claim deployment completion. A cancellation stays terminal unless the user
asks to reopen it. A failed terminal notification requires current-state reads before reporting; do not blindly replay. Autoscale (`on_demand`) is different: explain its idle-sleep and cold-start
risks verbally, obtain explicit user agreement, and then issue its direct hosting PUT with no Widget.

Result	Stored configuration	Pending state

`409 secret_values_required`	Candidate declarations are not yet applied.	Protected-value input has been opened.
`409 confirmation_required`	Requested change is not yet applied.	Owner confirmation, such as GitHub creation, is pending.
`409 hosting_confirmation_required`	Requested Reserved hosting change is not yet applied.	Reserved Widget is open and pauses the agent.
Input/confirmation completion	The original pending change is applied as a whole.	Terminal; Host/Widget notifies the Session.
Cancellation / expiry	Prior applied configuration is retained.	Terminal; `confirmation_cancelled` / `confirmation_expired` can identify this outcome.

Completion needs no extra settlement call or repeat of the original write. A new replacement
request after completion is a new operation; per-key identity is defined in secrets.

While protected input is open, `pending_secret_input` reports the collected keys, `blocks_config_writes`, `expires_at`, and `unblock` (a declaration-withdrawal DELETE). `false` allows unrelated scoped Web updates while input is pending; conflicting keys, changes
that invalidate the pending input, and whole-document replacement return `409 secret_input_pending`.
`true` identifies a legacy candidate that still blocks all config writes.
Validation precedes the input gate, so `422` does not prove there is no pending card. Empty forms have no deadline; staging the first value starts a 15-minute deadline. Use returned `unblock` paths verbatim to withdraw unwanted input. Deleting a pending-only key can succeed
without advancing revision. Otherwise wait for the required input. Do not reopen cancelled cards unless requested or replay completed replacements; use GET config to refresh runtime.

`error.reason_code: secret_input_in_progress` names this pending input state even when Stripe enablement reports it under `revision_conflict`. It is not concurrent revision movement.

Stored and active state

Successful config responses with `revision` also report `runtime_sync`: `applied` loaded that stored revision, `noop` means it was already loaded, and `failed: <reason>` means loading failed

while previous active runtime values remained. A successful `GET config` attempts loading again. Rejections and revisionless responses such as `GET infra/overview` omit this field; omission is not a synchronization failure.

Runtime configuration does not start or restart application processes. Environment updates reach subsequent commands, including commands in existing terminals; already-running processes
retain their startup environment. Value delivery has additional secret-specific constraints. The effect of a saved domain belongs to its page: runtime, features, hosting, or canonical Git.
Provider, region, and scaling policy are platform choices, not config fields.

The `pwa` domain controls published manifest ownership. It reaches the published site within about 60 seconds without republishing; see PWA.